

Data Structures



Lecture 5 Linked List Functions (Continued)

Prantik Paul [PNP]

Lecturer

Department of Computer Science and Engineering
BRAC University

Linked List - Search Element

```
1 # Getting index of an specific element
2 def indexOf(head, elem):
3     temp = head
4     count = 0
5     while temp != None:
6         if elem == temp.elem:
7             return count
8         count += 1
9         temp = temp.next
10    return -1 # Here -1 represents the absence of element in the list
```

Linked List - Search Element

```
1 def contains(head, elem):  
2     temp = head  
3     while temp != None:  
4         if elem == temp.elem:  
5             return True  
6         temp = temp.next  
7     return False
```

Linked List - Insert Element

```
1 def insert(head, elem, idx):
2     total_nodes = count(head)
3     if idx == 0: # Inserting at the beginning
4         n = Node(elem, head)
5         head = n
6     elif idx >= 1 and idx < total_nodes: # Inserting at the middle
7         n = Node(elem, head)
8         n1 = nodeAt(head, idx - 1)
9         n2 = nodeAt(head, idx)
10        n.next = n2
11        n1.next = n
12    elif idx == total_nodes: # Inserting at the end
13        n = Node(elem, None)
14        n1 = nodeAt(head, total_nodes - 1)
15        n1.next = n
16    else:
17        print("Invalid Index")
18    return head
```

Linked List - Remove Element

```
1 def remove(head, idx):
2     if idx == 0: # Removing first element
3         head = head.next
4     elif idx >= 1 and idx < count(head): # Removing middle element
5         n1 = nodeAt(head, idx - 1)
6         removed_node = n1.next
7         n1.next = removed_node.next
8     else:
9         print("Invalid Index")
10    return head
```

Linked List - Copy List

```
1 def copyList(source):
2     copy_head = None
3     copy_tail = None
4     temp = source
5     while temp != None:
6         n = Node(temp.elem, None)
7         if copy_head == None:
8             copy_head = n
9             copy_tail = copy_head
10        else:
11            copy_tail.next = n
12            copy_tail = copy_tail.next
13        temp = temp.next
14    return copy_head
```

Linked List - Reverse List (Out of Place)

```
1 def reverse_out_of_place(head):  
2     new_head = Node(head.elem, None)  
3     temp = head.next  
4     while temp != None:  
5         n = Node(temp.elem, new_head)  
6         new_head = n  
7         temp = temp.next  
8     return new_head
```


Linked List - Reverse List (In Place)

```
1 def reverse_in_place(head):  
2     new_head = None  
3     temp = head  
4     while temp != None:  
5         n = temp.next  
6         temp.next = new_head  
7         new_head = temp  
8         temp = n  
9     return new_head
```


Linked List - Rotate Left

```
1 def rotate_left(head):  
2     new_head = head.next  
3     temp = new_head  
4     while temp.next != None:  
5         temp = temp.next  
6     temp.next = head  
7     head.next = None  
8     head = new_head  
9     return head
```

Linked List - Rotate Right

```
1 def rotate_right(head):
2     last_node = head.next
3     second_last_node = head
4     while last_node.next != None:
5         last_node = last_node.next
6         second_last_node = second_last_node.next
7     last_node.next = head
8     second_last_node.next = None
9     head = last_node
10    return head
```